



哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY



大模型推理服务(I)

Batching, PagedAttention, RadixAttention

王宏志

哈尔滨工业大学

计算学部 海量数据计算研究中心

wangzh@hit.edu.cn

<http://homepage.hit.edu.cn/wang>

目录

Contents

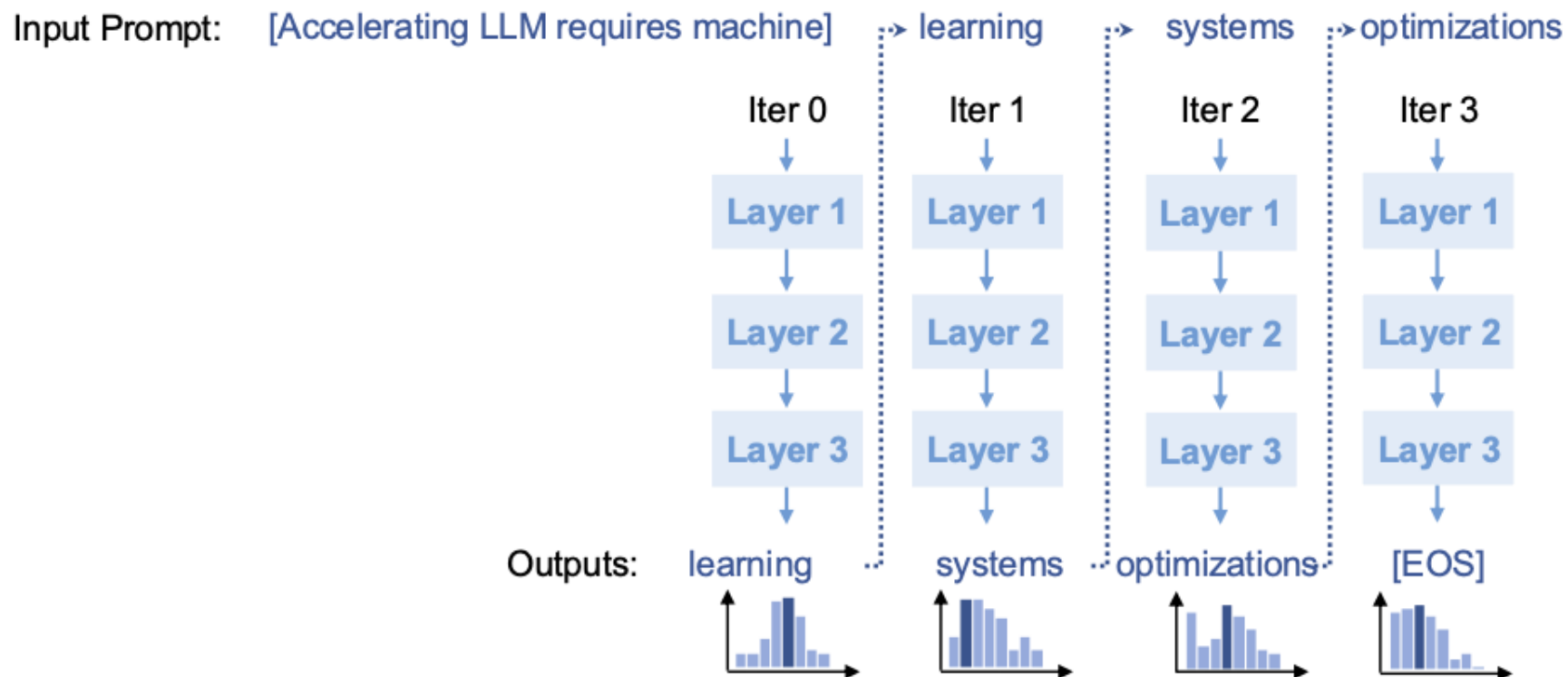
- 1 连续批处理
- 2 PagedAttention 与KV Cache
- 3 RadixAttention 与前缀共享
- 4 调度优化与 CPU 开销隐藏

目录

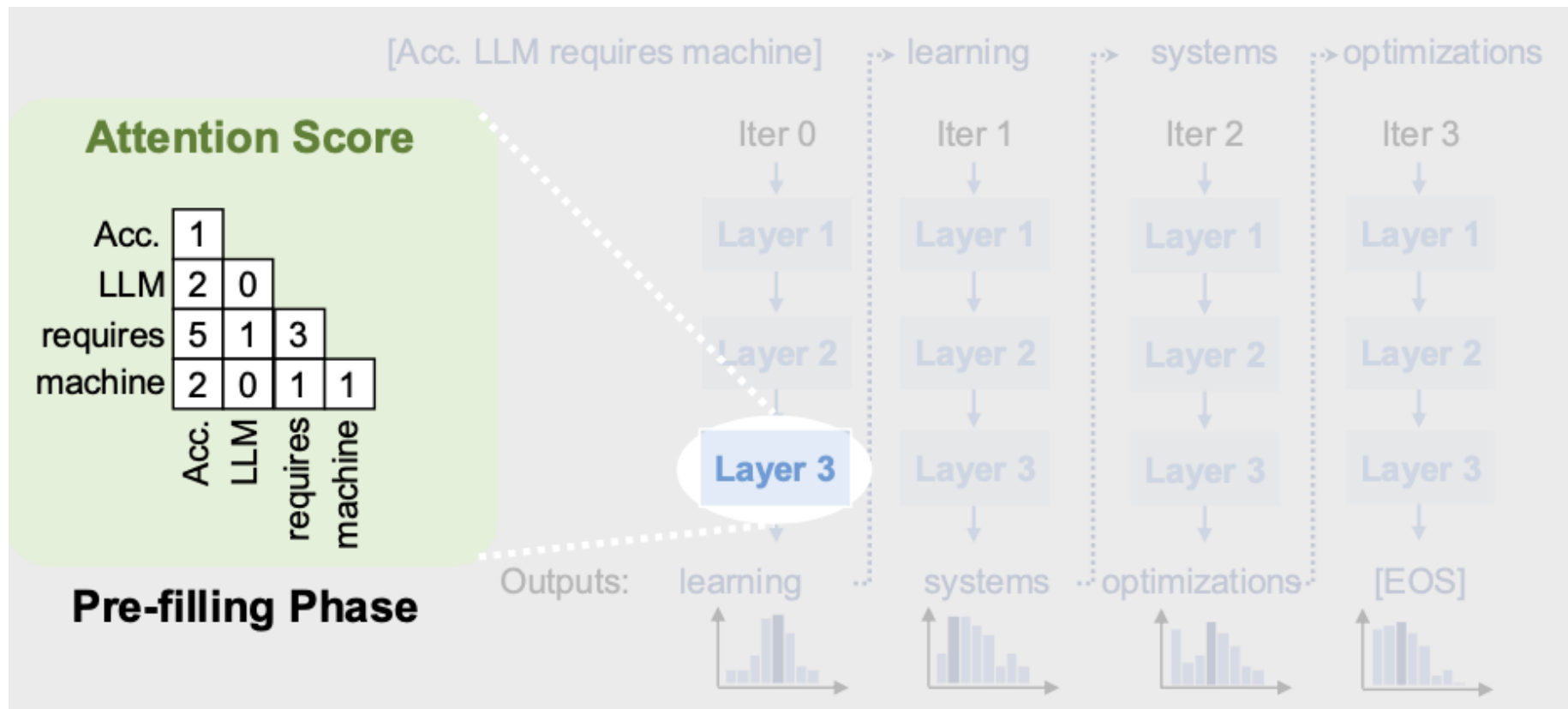
Contents

- 1 连续批处理
- 2 PagedAttention 与KV Cache
- 3 RadixAttention 与前缀共享
- 4 调度优化与 CPU 开销隐藏

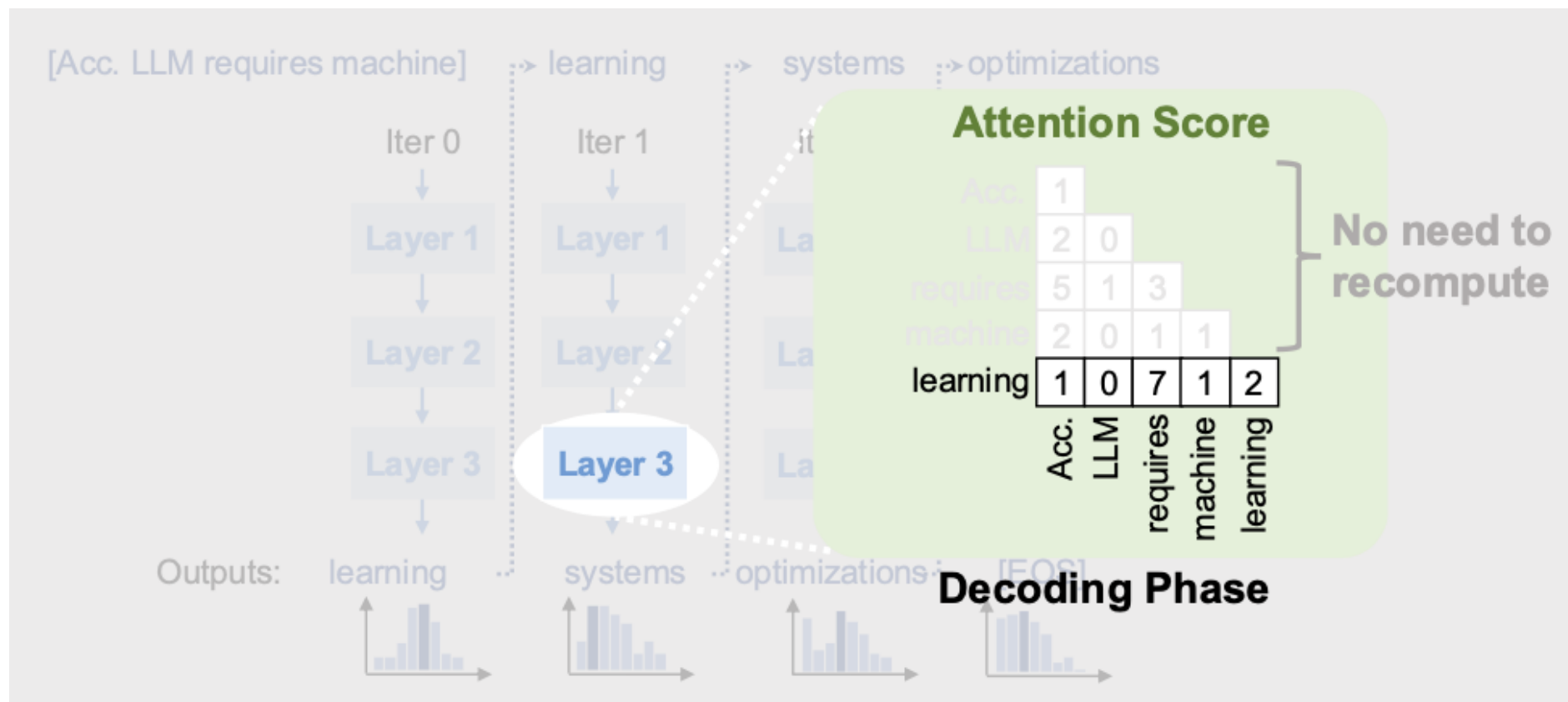
生成式 LLM 推理:自回归解码



生成式 LLM 推理:自回归解码



生成式 LLM 推理:自回归解码



生成式 LLM 推理:自回归解码

- 预填充阶段（0次迭代）：
 - 一次性处理所有输入标记
- 解码阶段（所有其他迭代）：
 - 处理从上一迭代生成的单个标记
 - 使用所有先前标记的注意力键和值
- Key-value缓存:
 - 保存注意力键和值以供后续迭代使用,以避免重新计算

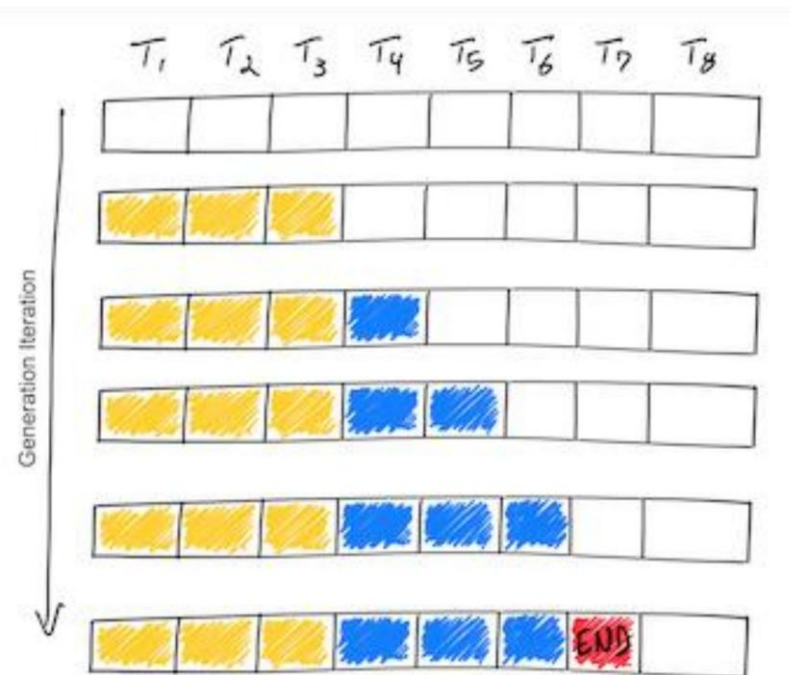
LLM 服务缓慢且昂贵



- 至少需要10个A100-40GB GPU来服务175B GPT-3的半精度计算
- 生成256个Token需要约20秒
- 不能并行处理许多请求
 - 每请求键值缓存占用 3GB GPU 内存

- 持续分批
- 分页注意力
- 基数注意力

LLM Decoding Timeline



批处理请求以提高 GPU 性能

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3	S_3				
S_4	S_4	S_4	S_4	S_4			

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END		
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END			
S_4	S_4	S_4	S_4	S_4	S_4	END	

静态批处理的问题:

- 请求可以在不同的迭代中完成
- 空闲 GPU 周期
- 新请求不能立即开始

连续批处理

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3	S_3				
S_4	S_4	S_4	S_4	S_4			

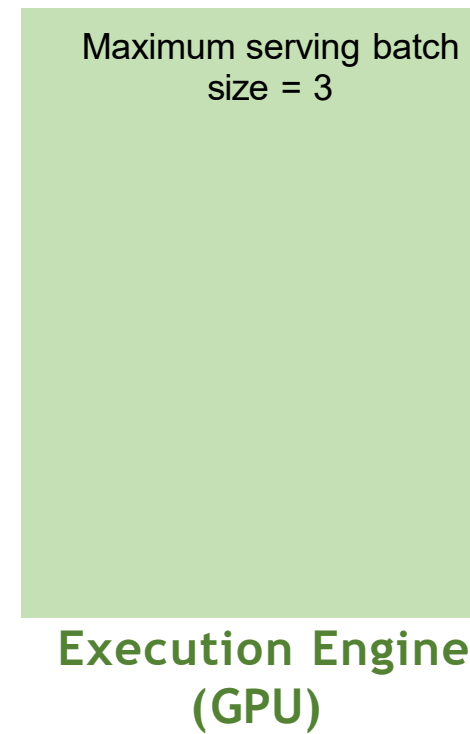
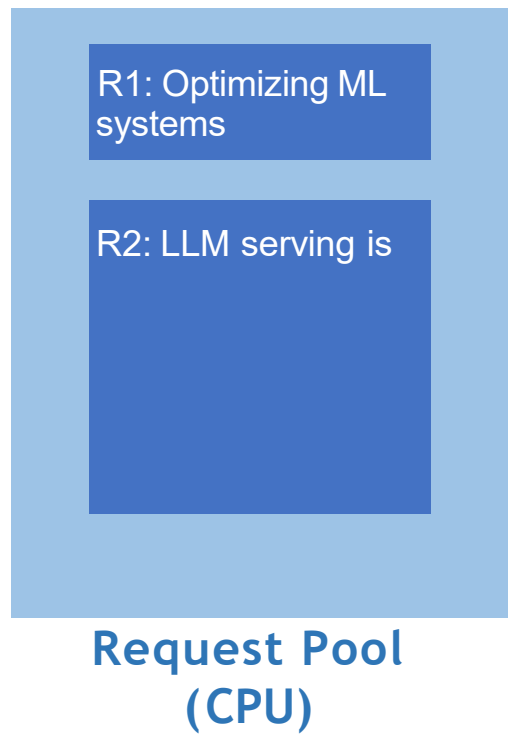
T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

好处:

- 更高的 GPU 利用率
- 新请求可以立即开始

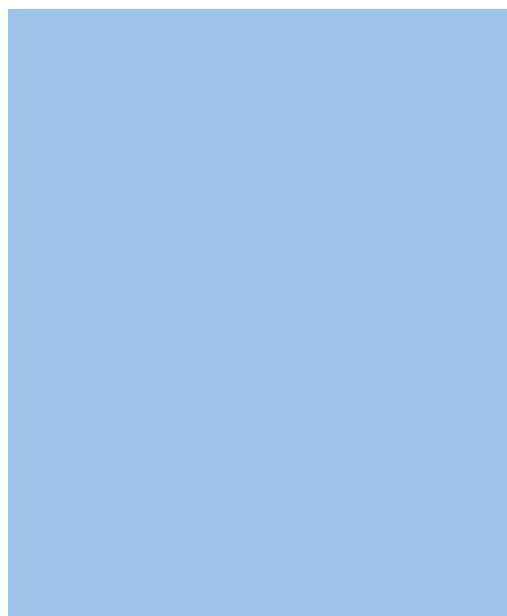
连续分批步骤

- 收到两个新请求 R1 和 R2

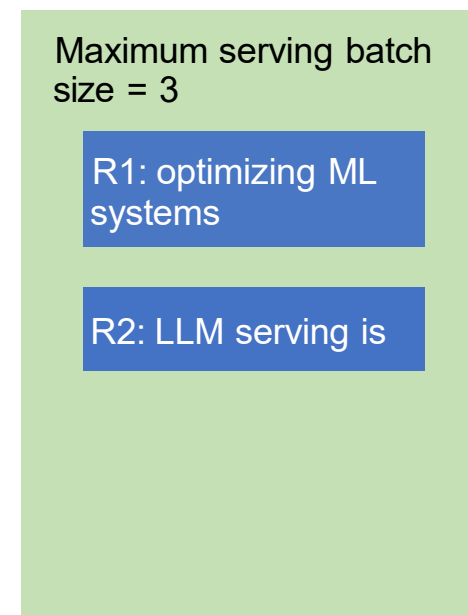


连续分批步骤

• 迭代 1：解码 R 1 和 R 2



Request Pool
(CPU)



Execution Engine
(GPU)

C

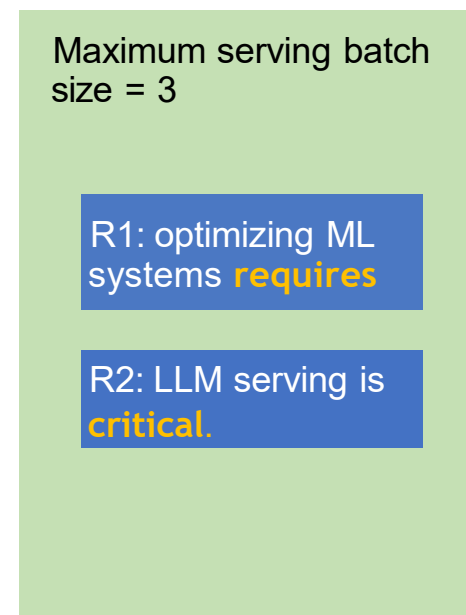
Iteration 1

连续分批步骤

- 收到新请求 R3；完成解码 R1 和 R2



Request Pool
(CPU)



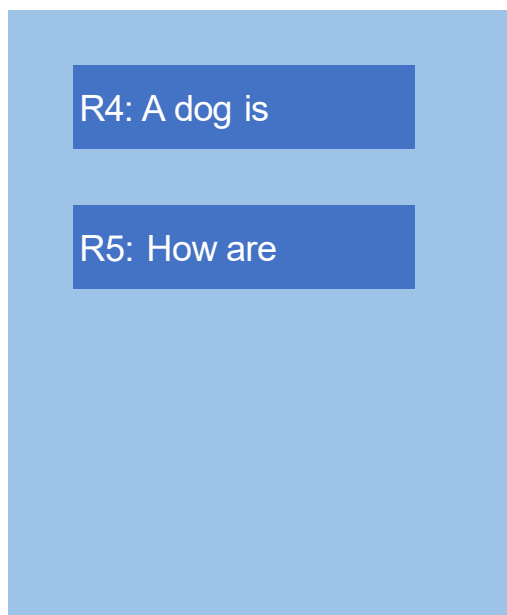
Execution Engine
(GPU)

C

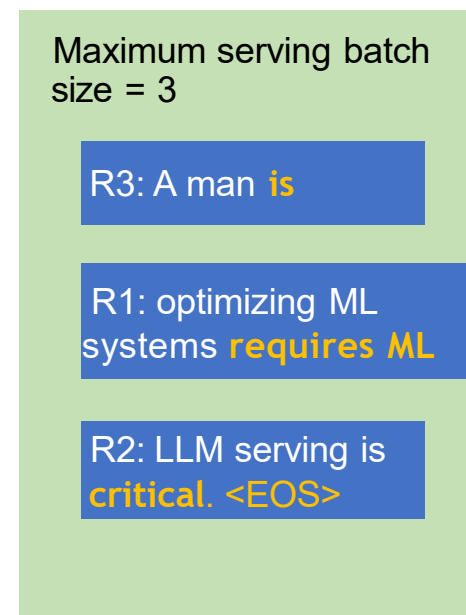
Iteration 1

连续分批步骤

- 迭代2：解码 R1、R2、R3；接收 R4、R5；R2 完成



Request Pool
(CPU)



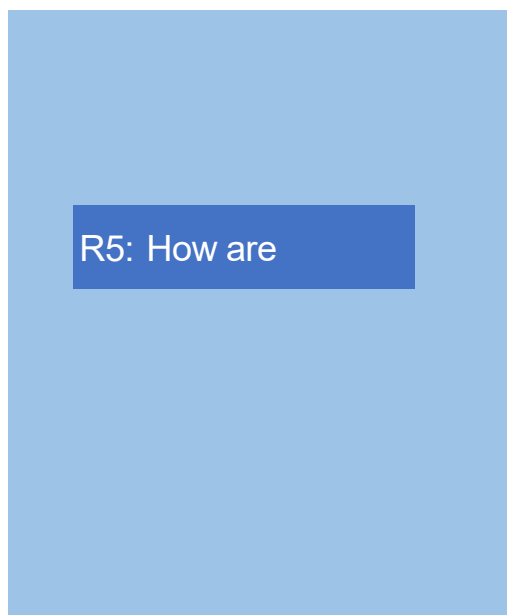
Execution Engine
(GPU)

C

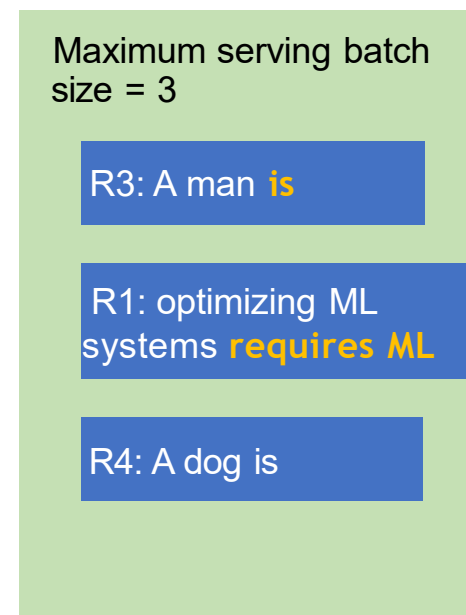
Iteration 2

连续分批步骤

•迭代3：解码 R 1 , R 3 , R 4



Request Pool
(CPU)



Execution Engine
(GPU)

C

Iteration 3

连续批处理

- 更高效地处理提前完成和迟到请求
- 更高的GPU利用率

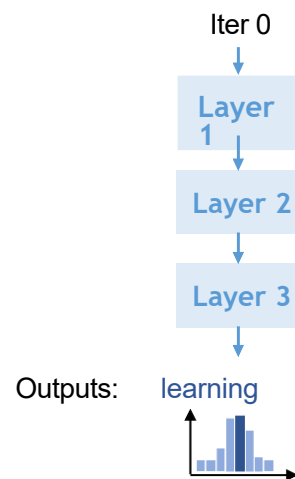
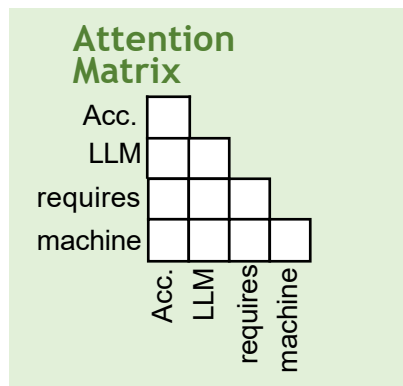
目录

Contents

- 1 连续批处理
- 2 **PagedAttention 与KV Cache**
- 3 RadixAttention 与前缀共享
- 4 调度优化与 CPU 开销隐藏

KV缓存动态增长和收缩

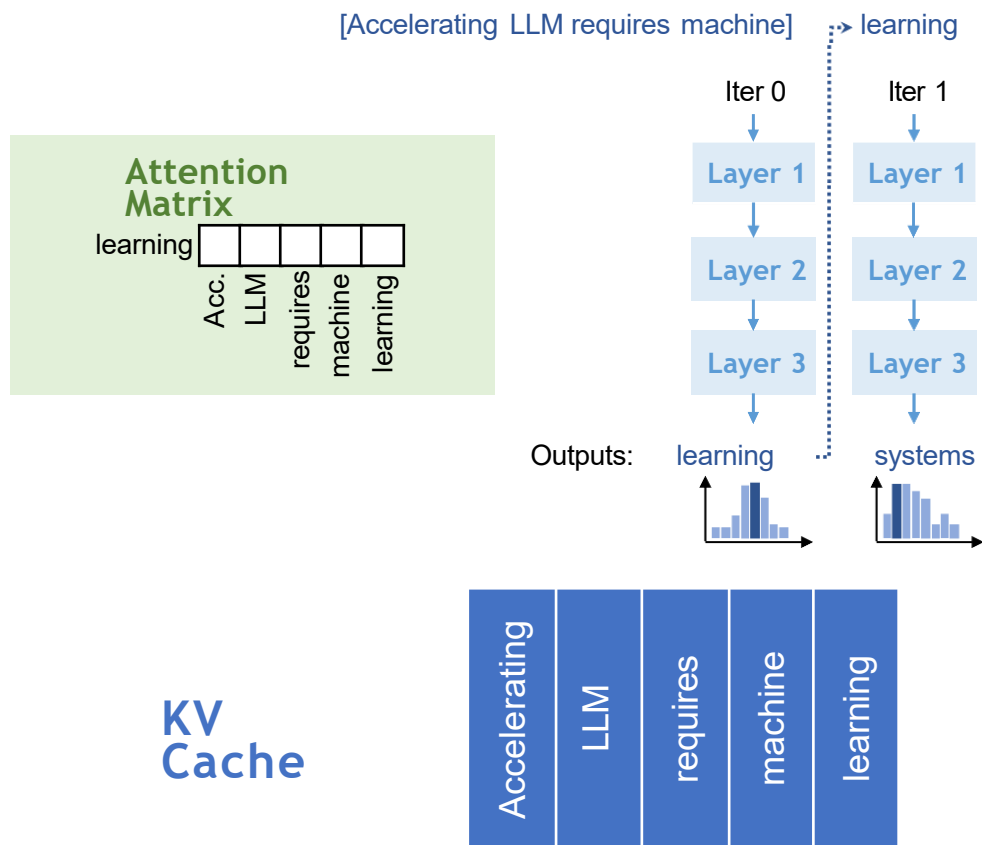
[Accelerating LLM requires machine]



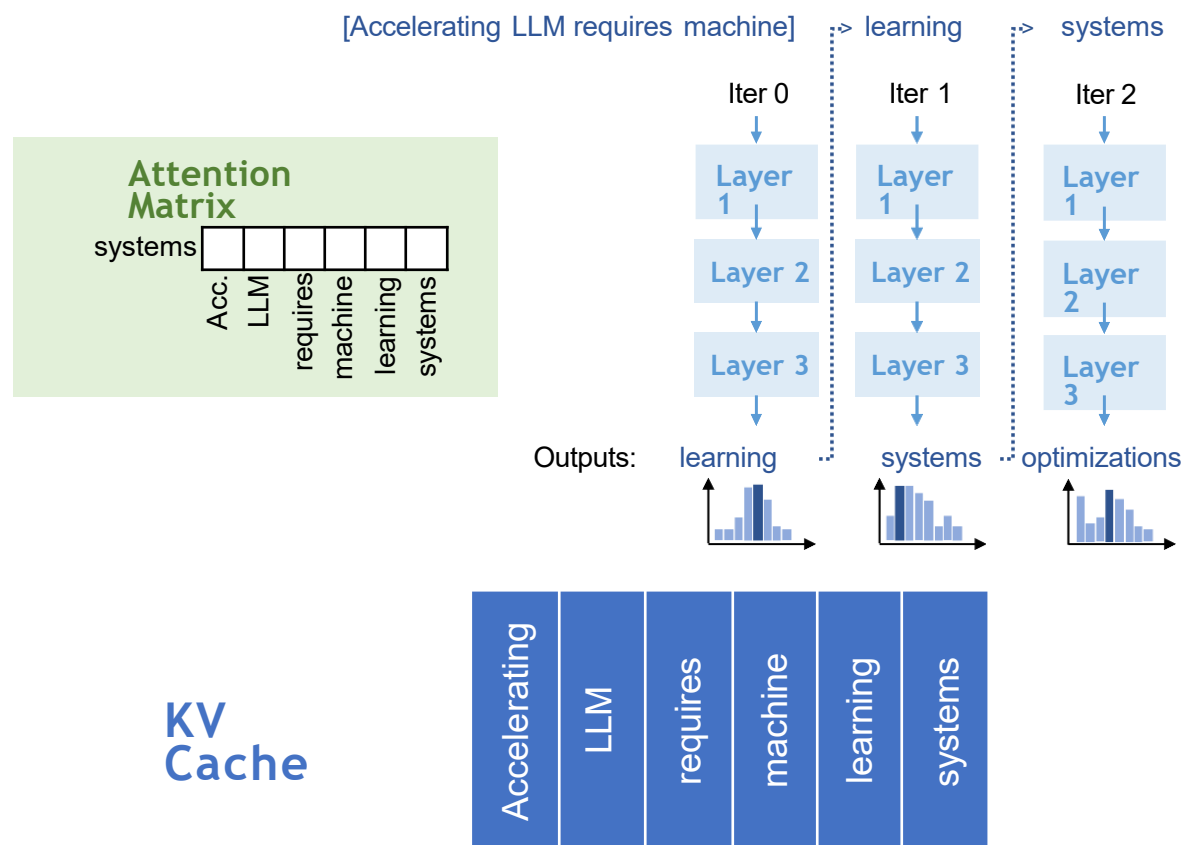
**KV
Cache**



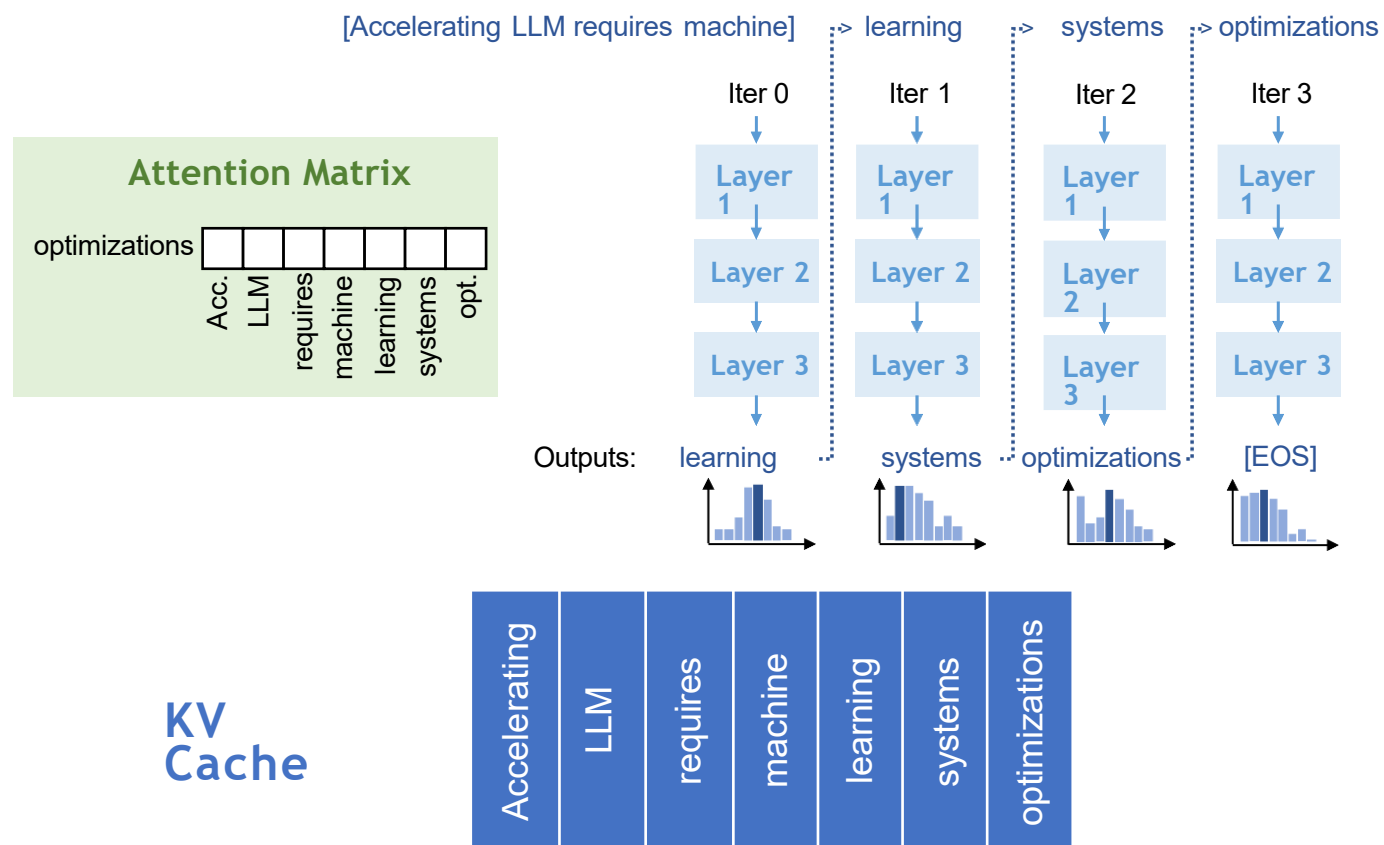
KV缓存动态增长和收缩



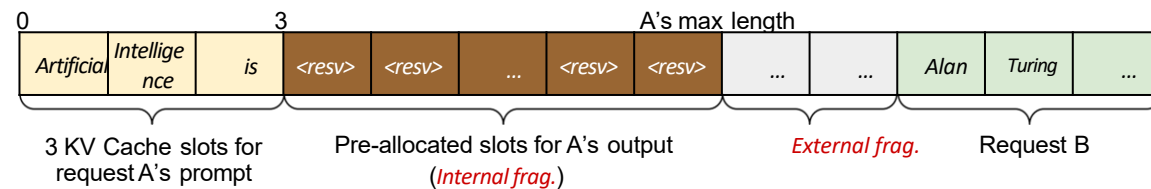
KV缓存动态增长和收缩



KV缓存动态增长和收缩



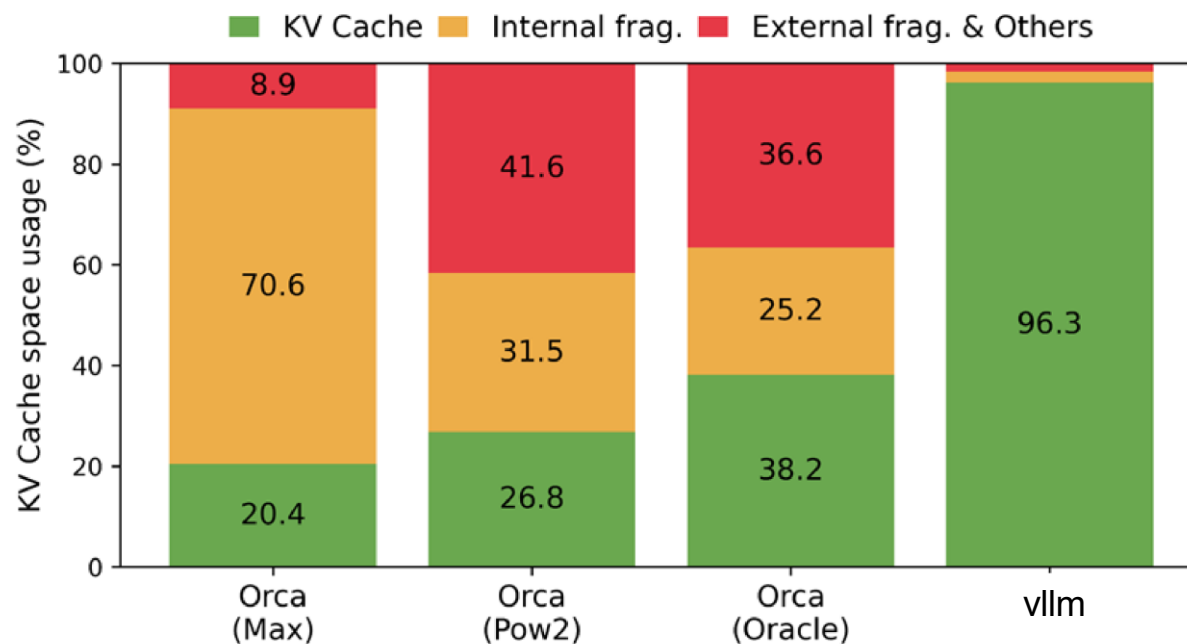
静态KV缓存管理浪费内存



- 预先为请求的最大长度分配连续的内存空间
- 内存碎片
 - 由于未知输出长度导致的内部碎片
 - 由于非均匀的每请求最大长度导致的外部碎片化

KV缓存中的显著内存浪费

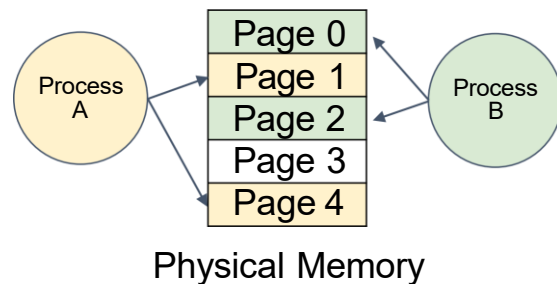
- 仅利用20-40%的KV缓存来存储实际Token状态



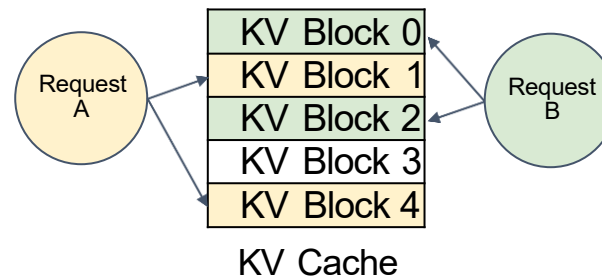
PagedAttention

- 内存分页和虚拟化技术用于KV缓存

Memory management in OS

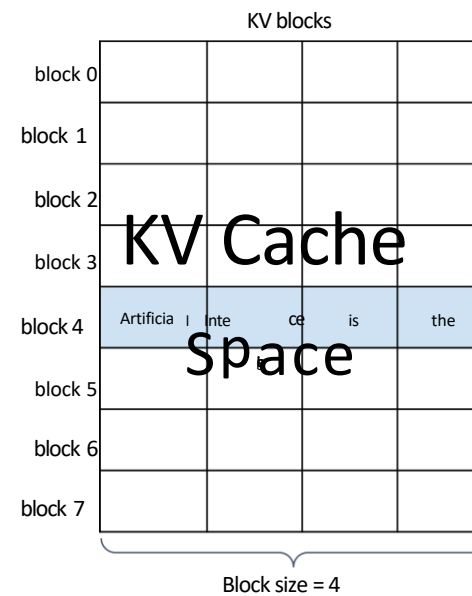


PagedAttention



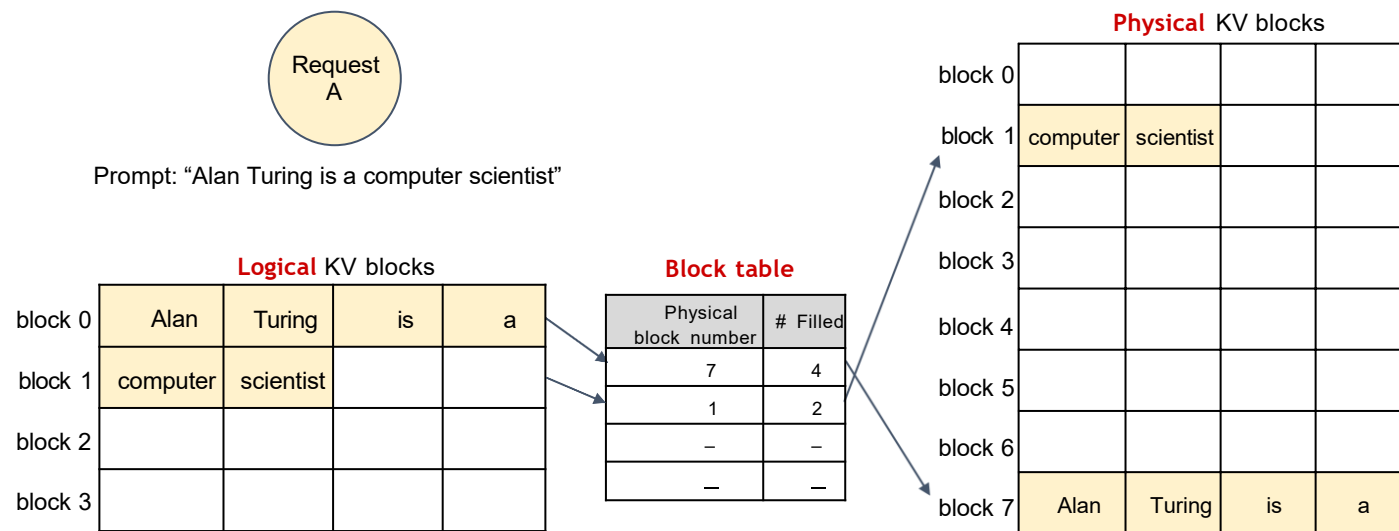
分页KV缓存空间到KV块*

- KV块是存储从左到右的KV状态的固定大小连续内存块



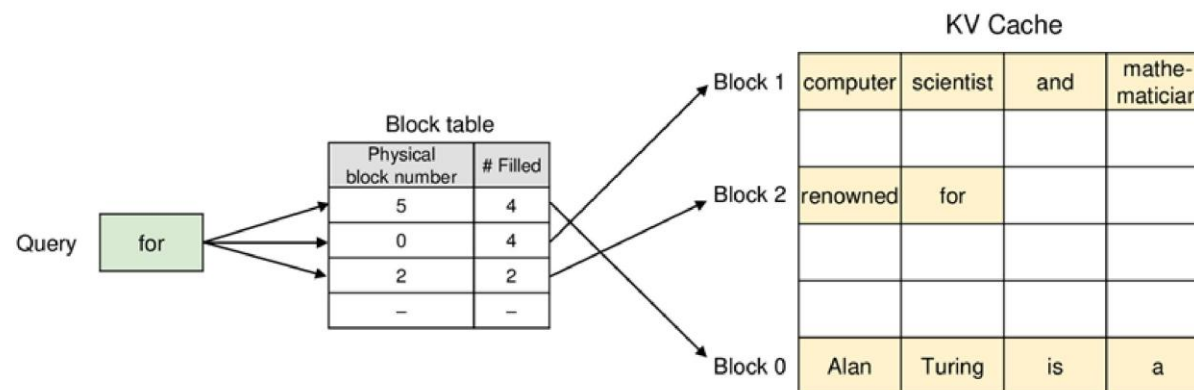
“block”这个术语在PagedAttention中是过载的。

虚拟化KV缓存



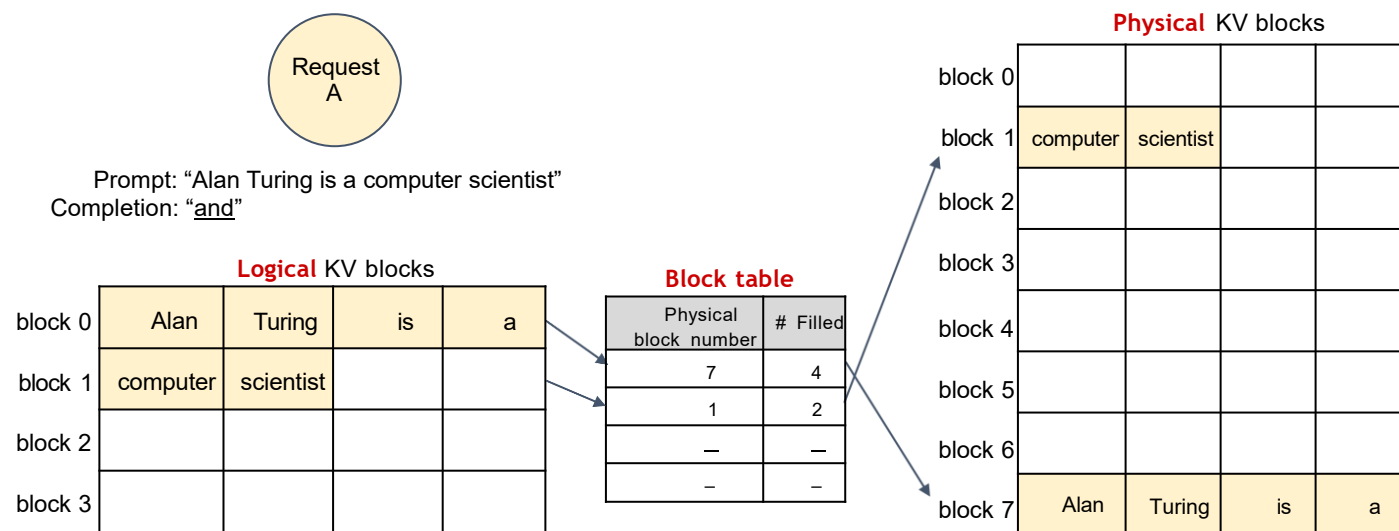
请注意虚拟化KV缓存

1. 使用块表获取非连续的KV块
2. 行中应用注意力

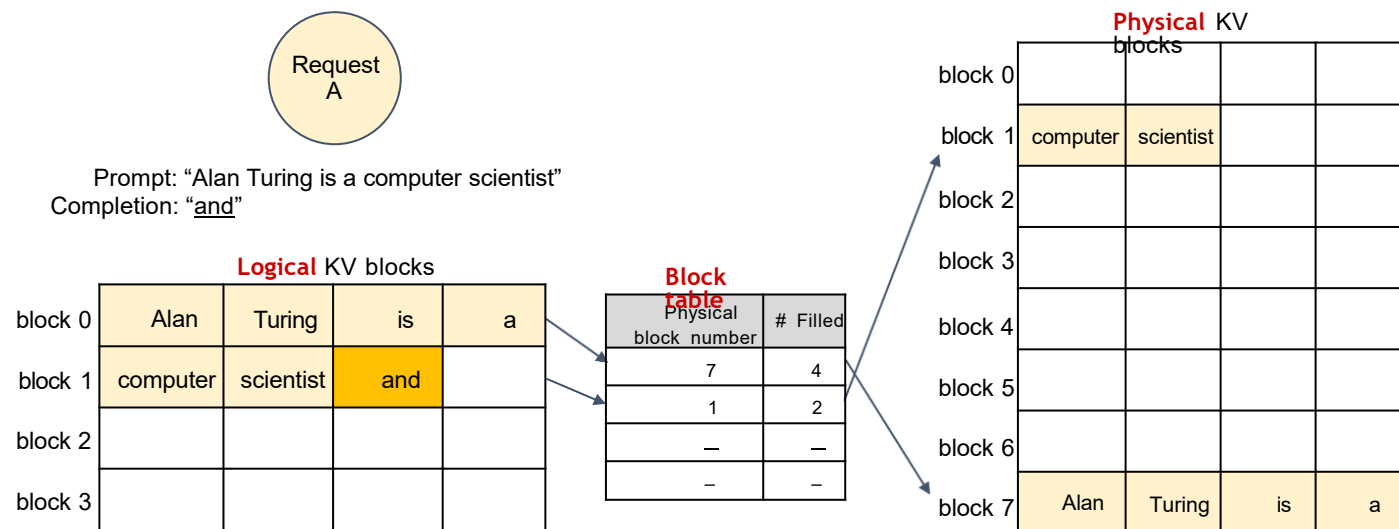


关键洞察：注意力是关联和可交换的

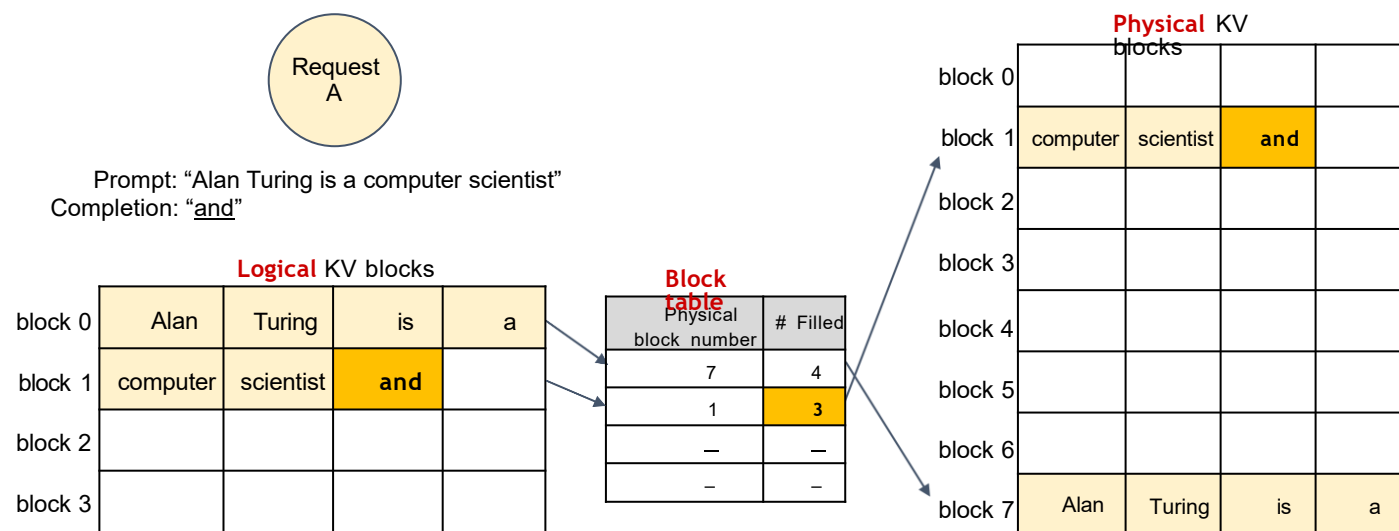
分页注意力内存管理



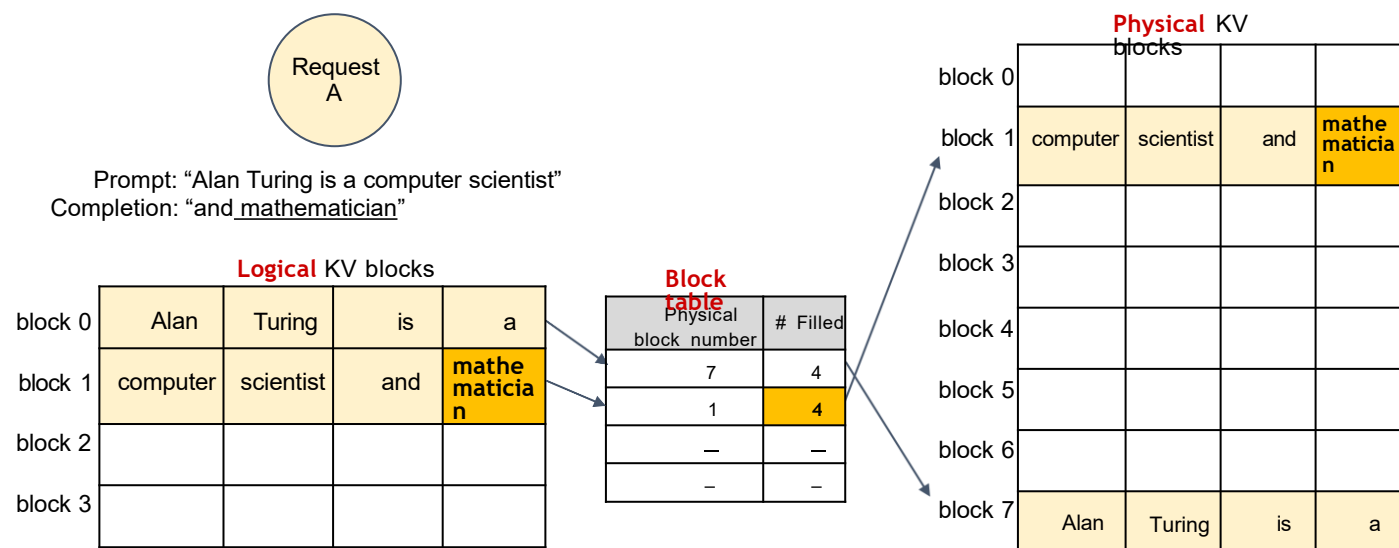
分页注意力内存管理



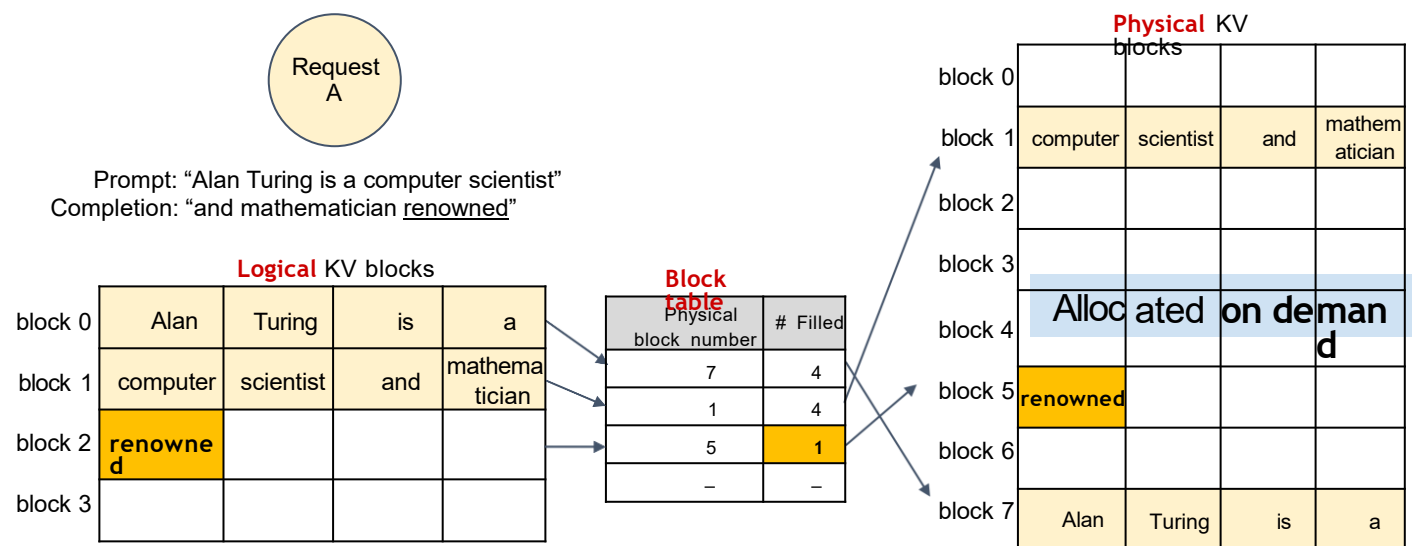
分页注意力内存管理



分页注意力内存管理



分页注意力内存管理



分页注意力机制的内存效率

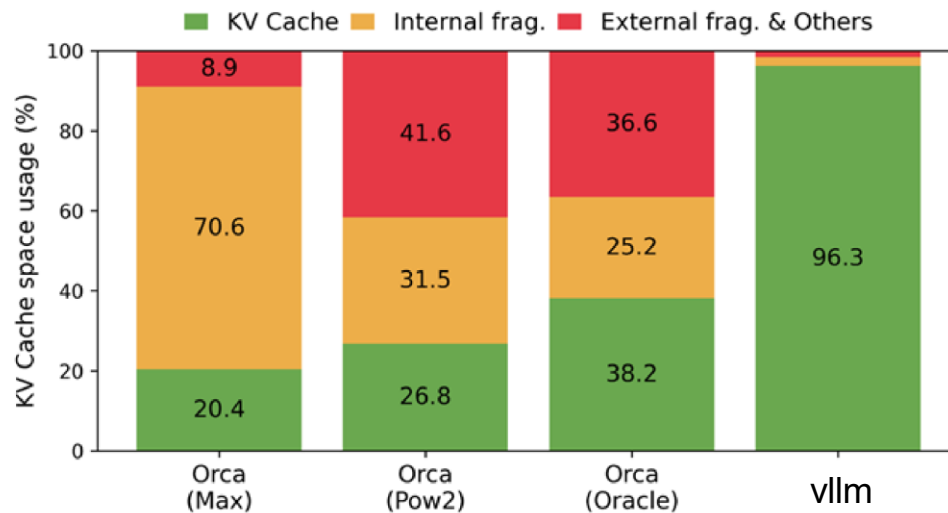
最小内部碎片

- 仅发生在序列的最后一块
- $\# \text{ 浪费Token} / \text{序列} < \text{块大小}$

无外部碎片

Alan	Turing	is	a
computer	scientist	and	mathemat i cian
renowned			

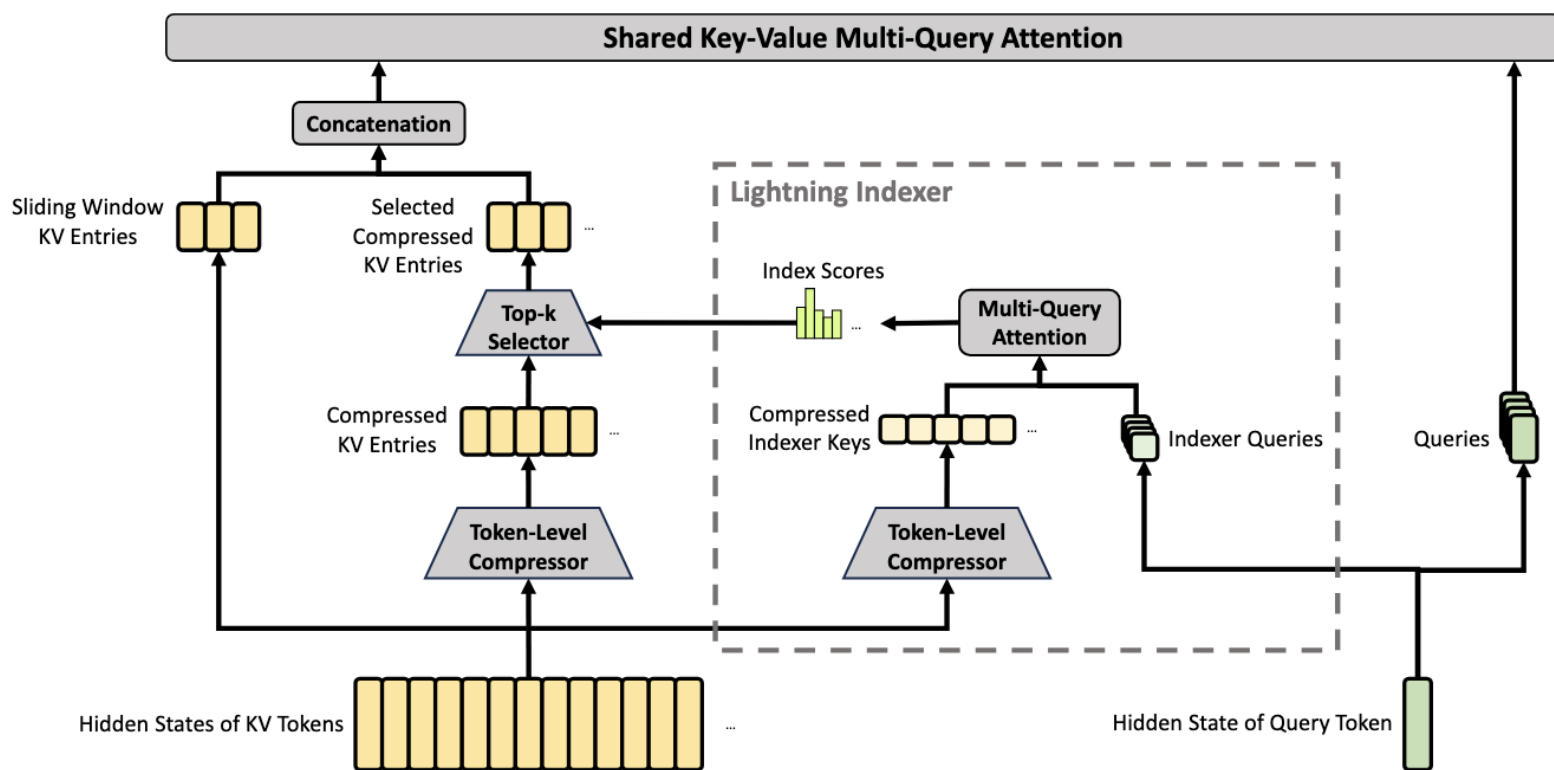
Internal
fragmentation



压缩稀疏注意力 (CSA)

压缩稀疏注意力 (CSA)

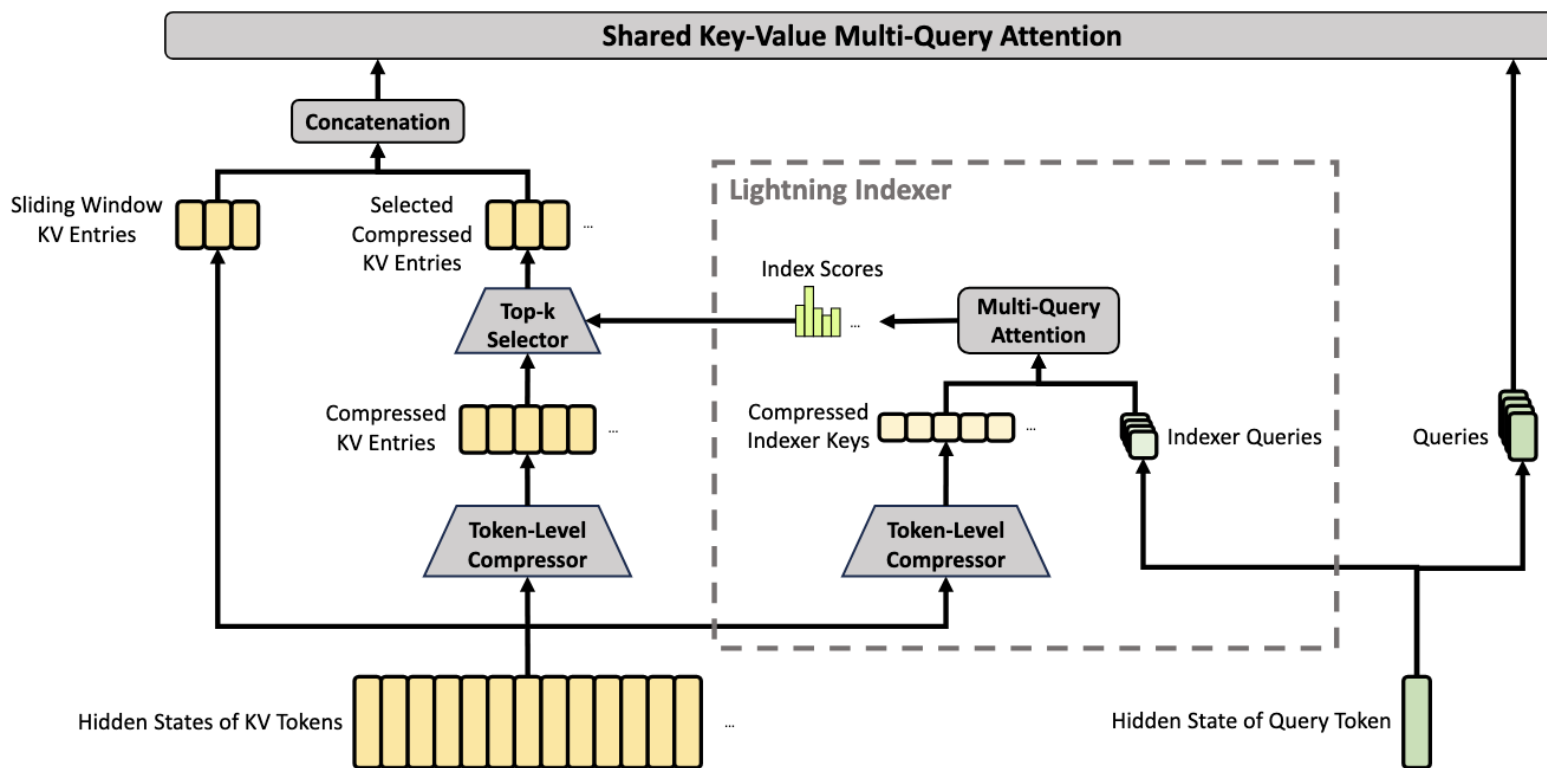
它首先将每 m 个Token的 KV 缓存压缩为一个条目，然后应用深度探索(DeepSeek)稀疏注意力进行进一步加速。



压缩稀疏注意力 (CSA)

✓ 压缩键值条目

设 $H \in R_{n \times d}$ 为输入隐藏状态的序列，其中 n 是序列长度， d 是隐藏大小。CSA 首先计算两个系列的 KV 条目 $C_a, C_b \in R_{n \times c}$ 及其相应的压缩权重 $Z_a, Z_b \in R_{n \times c}$ ，其中 c 是头



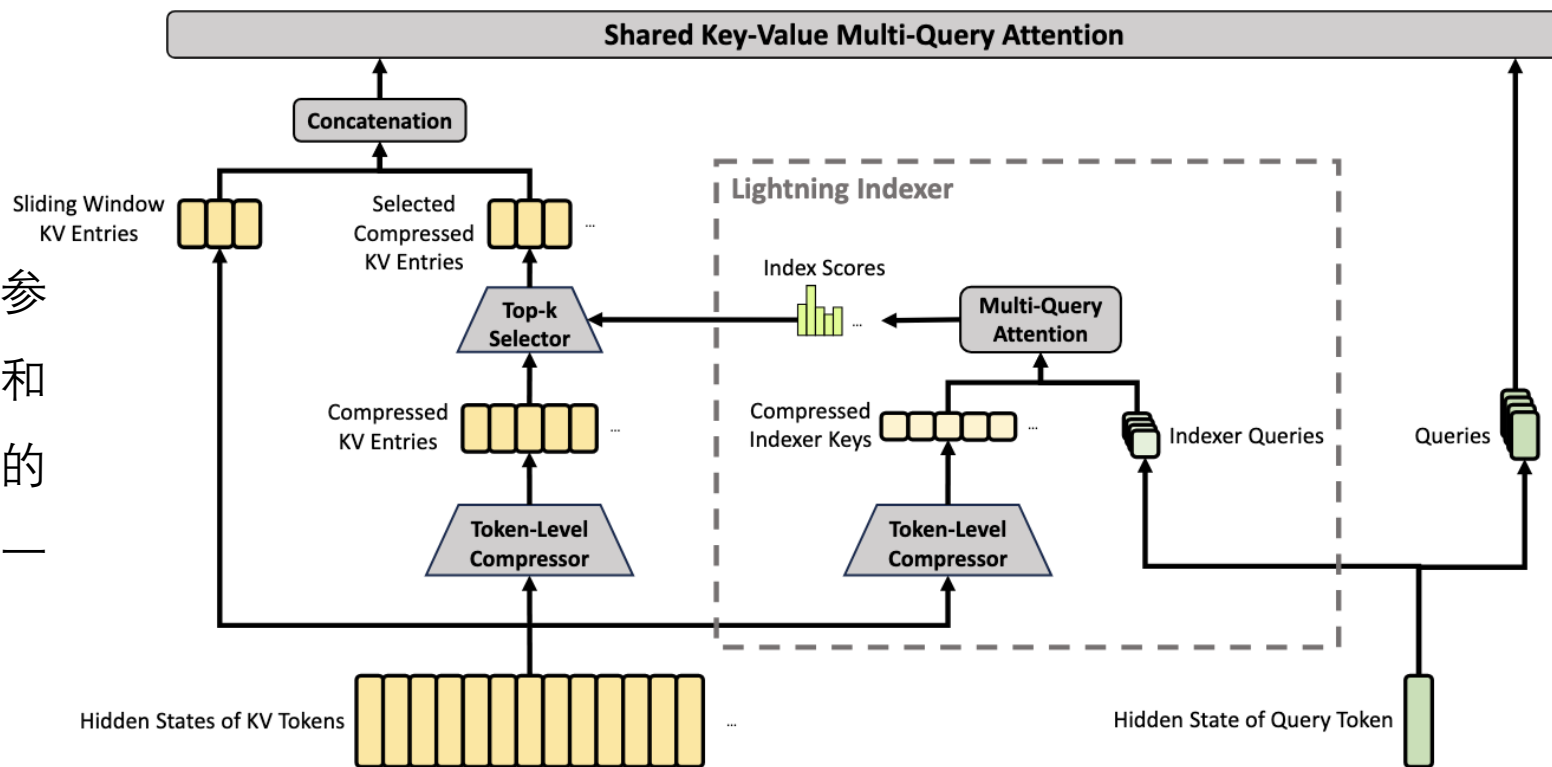
压缩稀疏注意力 (CSA)

✓ 压缩键值条目

设 $H \in R_{n \times d}$ 为输入隐藏状态的序列，其中 n 是序列长度， d 是隐藏大小。CSA 首先计算两个系列的 KV 条目 $C_a, C_b \in R_{n \times c}$ 及其相应的压缩权重 $Z_a, Z_b \in R_{n \times c}$ ，其中 c 是头部维度

$$C^a = H \cdot W^{aKV}, \quad C^b = H \cdot W^{bKV},$$
$$Z^a = H \cdot W^{aZ}, \quad Z^b = H \cdot W^{bZ},$$

$W^{aKV}, W^{bKV}, W^{aZ}, W^{bZ} \in R^{d \times c}$ 是可训练的参数。接下来，每个 m KV 条目在 C_a 和 C_b 中将根据它们的压缩权重和可学习的位置偏差 $B^a, B^b \in R^{m \times c}$ 被压缩成一个条目，产生 $C_i^{Comp} \in R^{\frac{n}{m} \times c}$ 。



压缩稀疏注意力 (CSA)

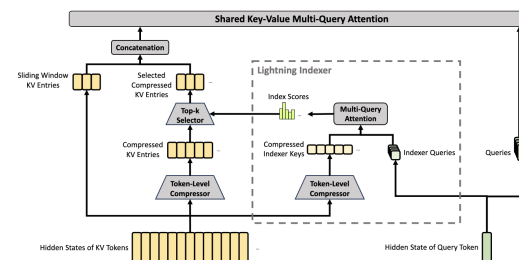
✓ 压缩键值条目

每个压缩条目 $C_i^{\text{Comp}} \in R^c$ 是通过计算

$$[S_{mi:m(i+1)-1}^a; S_{m(i-1):mi-1}^b] = \text{Softmax}_{\text{row}}([Z_{mi:m(i+1)-1}^a + B^a; Z_{m(i-1):mi-1}^b + B^b]),$$
$$C_i^{\text{Comp}} = \sum_{j=mi}^{m(i+1)-1} S_j^a \odot C_j^a + \sum_{j=m(i-1)}^{mi-1} S_j^b \odot C_j^b,$$

其中, $\odot$ 表示 Hadamard 乘积; $\text{Softmax}_{\text{row}}(\cdot)$ 表示沿行维度执行的 softmax 操作, 对来自 Z^a 和 Z^b 的 $2m$ 个元素进行归一化。当 $i = 0$ 时, $Z_{m(i-1):mi-1}^b$ 被填充为负无穷大, 而 $C_{m(i-1):mi-1}^b$ 被填充为零。

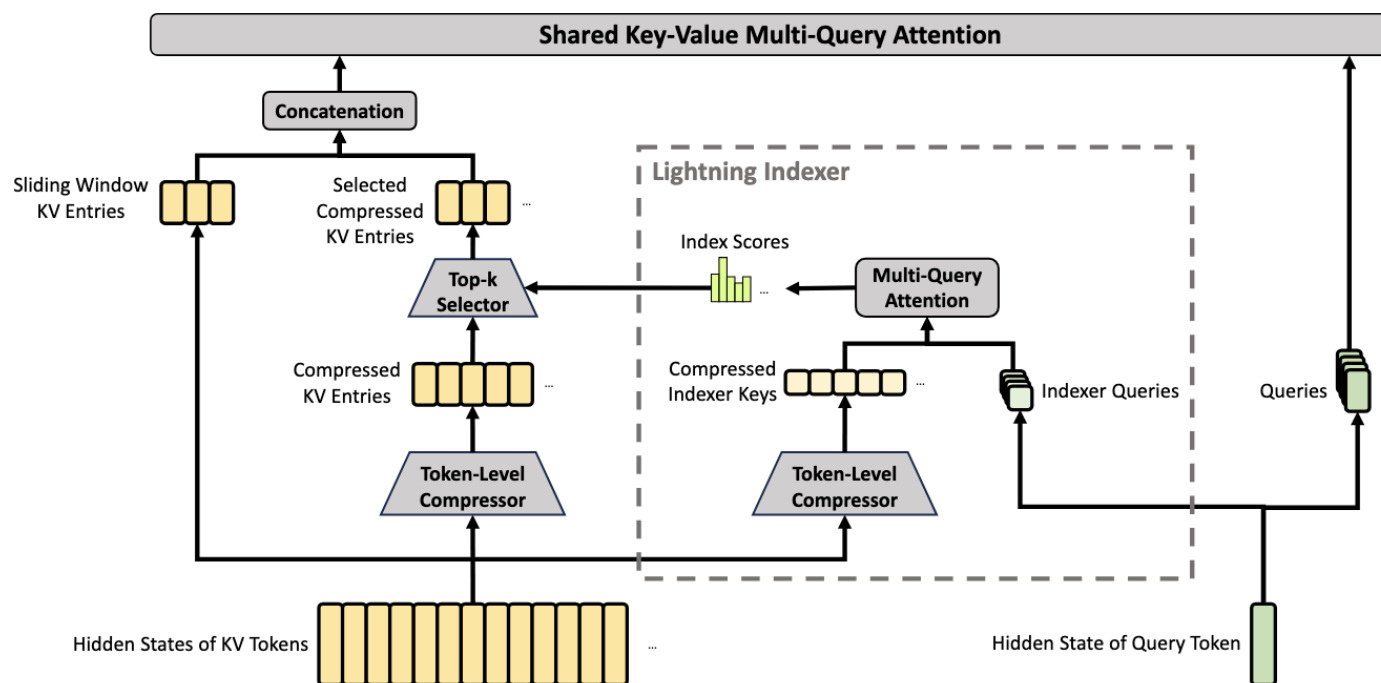
注意, 每个 C_i^{Comp} 是从 $2m$ 个 KV 条目中得出的, 但用于 C^{Comp}_i 的 C^b 的索引和用于 C^{Comp}_{i-1} 的 C^a 的索引是重叠的。因此, CSA 实际上将序列长度压缩为原来的 $1/m$ 。



压缩稀疏注意力 (CSA)

✓ 稀疏选择的 Lightning Indexer

在获得压缩后的 KV 条目 C^{Comp} 后，CSA 应用 DSA 策略选择 top-k 个压缩的 KV 条目用于核心注意力。首先，CSA 对 C^{Comp} 执行相同的压缩操作，以获得压缩后的索引器键 $k_{IComp} \in \mathbb{R}^{c'}$ ，其中 c' 是索引器的头维度。然后，对于查询Token t ，生成低秩的索引查询 $\{q'_{1,1}, q'_{2,1}, \dots, q'_{l',n}\}$ 。



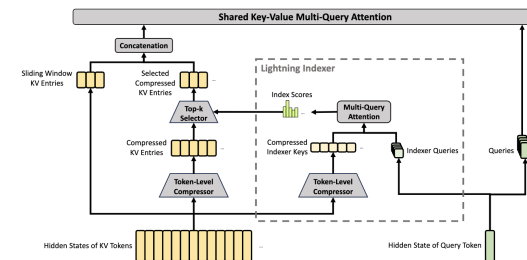
压缩稀疏注意力 (CSA)

✓ 稀疏选择的 Lightning Indexer

在获得压缩后的 KV 条目 C^{Comp} 后, CSA 应用 DSA 策略选择 top-k 个压缩的 KV 条目用于核心注意力。首先, CSA 对 C^{Comp} 执行相同的压缩操作, 以获得压缩后的索引器键 $k_{IComp} \in \mathbb{R}^{c'}$, 其中 c' 是索引器的头维度。然后, 对于查询Token t , 生成低秩的索引查询 $\{q'_{1,1}, q'_{2,1}, \dots, q'_{l',n}\}$ 。

$$\begin{aligned} \mathbf{c}_t^Q &= \mathbf{h}_t \cdot W^{DQ}, \\ [\mathbf{q}_{t,1}^I; \mathbf{q}_{t,2}^I; \dots; \mathbf{q}_{t,n_h^I}^I] &= \mathbf{q}_t^I = \mathbf{c}_t^Q \cdot W^{IUQ}, \end{aligned}$$

其中, $h_t \in \mathbb{R}^d$ 是查询Token t 的输入隐藏状态; $c_t^Q \in \mathbb{R}^d$ 是查询的压缩隐向量; d_c 表示查询压缩维度; n_l^I 表示索引查询头的数量; $W^Q \in \mathbb{R}^{d \times d_c}$ 和 $W^{UQ} \in \mathbb{R}^{d \times c'}$ 分别是索引查询的下投影和上投影矩阵。



压缩稀疏注意力 (CSA)

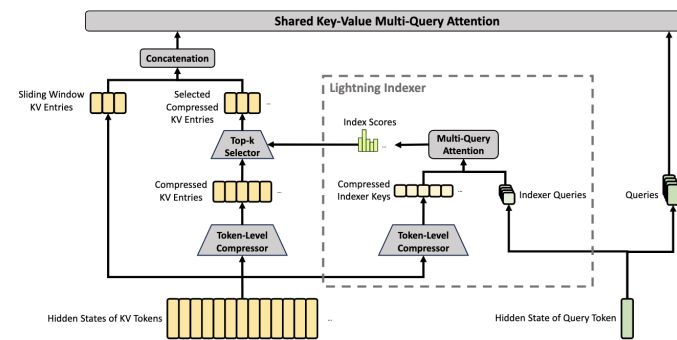
✓ 稀疏选择的 Lightning Indexer

接下来，索引分数 $I_t^s \in \mathbb{R}^{\mathbb{I}}$ 计算如下，其中 $s \leq \text{Floor}(m)$ ：

$$\left[w'_{1,1}, w'_{2,1}, \dots, w'_{n_l^I} \right] = h_t W^W I_t^s = \sum_{h=1}^{n_l^I} w_h^W \cdot \text{ReLU}(q_h^I, k_s^{IComp})$$

其中， $W^W \in \mathbb{R}^{d^{\mathbb{I}} \times m^{\mathbb{I}}}$ 是可学习的矩阵， $w_h^W \in \mathbb{R}$ 是第 h 个索引器的权重。对于给定查询Token t 和其索引分数 I_t^l ，使用 top-k 选择器来选择性地保留一部分压缩 KV 条目 $C_t^{SprsComp}$ ，以便用于后续的核心注意力：

$$C_t^{SprsComp} = \{C_t^{Comp} \mid I_t^s \in \text{Top-k}(I_t^l)\}$$



压缩稀疏注意力 (CSA)

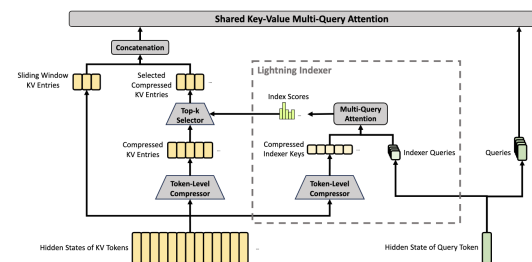
✓ 共享键值 MQA。

在选择稀疏的 KV 条目后，CSA 按照多查询注意力 (MQA) 的方式 (Shazeer, 2019) 执行注意力操作，其中每个压缩后的 KV 条目 $c_t^{SprsComp}$ 作为查询键和值。具体来说，对于查询Token t，首先从压缩的隐向量 c_t^Q 中提取注意力查询

其中， n_q 表示查询头的数量； $W^Q \in \mathbb{R}^{d_q \times m_q}$ 是查询的上投影矩阵。注意，最后一个查询向量 c_t^Q 与用于索引器查询的矩阵共享。接下来，对 $\{q_{t,i}\}$ 和 $c_t^{SprsComp}$ 执行 MQA 操作：

$$o_t^i = \text{CoreAttn}(\text{query} = q_{t,i}, \text{key} = c_t^{SprsComp}, \text{value} = c_t^{SprsComp})$$

其中， $o_t^i \in \mathbb{R}^d$ 是第 i 个头在第 t 个Token上的核心注意力输出； $\text{CoreAttn}(\cdot)$ 表示核心注意力操作。

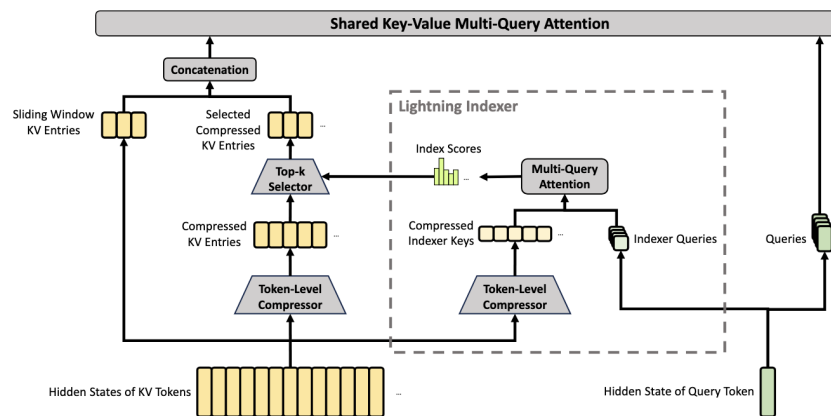


压缩稀疏注意力 (CSA)

✓ 分组输出投影。

在 DeepSeek-V4 的配置中，核心注意力的输出较大，因此直接投影核心注意力的输出 $[o_{t,1}, o_{t,2}, \dots, o_{t,n_h}] = o_t \in \mathbb{R}^d$ 会带来较大的计算开销。为了降低计算开销，设计了分组输出投影策略。具体来说，首先将输出 $o_{t,i}^g \in \mathbb{R}^{d_g}$ 投影到一个较小的维度空间 $\mathbb{R}^{d_g}$ ： $o_{t,i}^g = W^g o_{t,i}$ 。其中， $W^g \in \mathbb{R}^{d_g \times d}$ 是可学习的投影矩阵， $d_g < d$ 。

最后，将中间输出投影 $o_{t,i}^g \in \mathbb{R}^{d_g}$ 投影到最终的核心注意力输出 $\delta_t \in \mathbb{R}^d$ ： $\delta_t = W^\delta o_{t,i}^g$



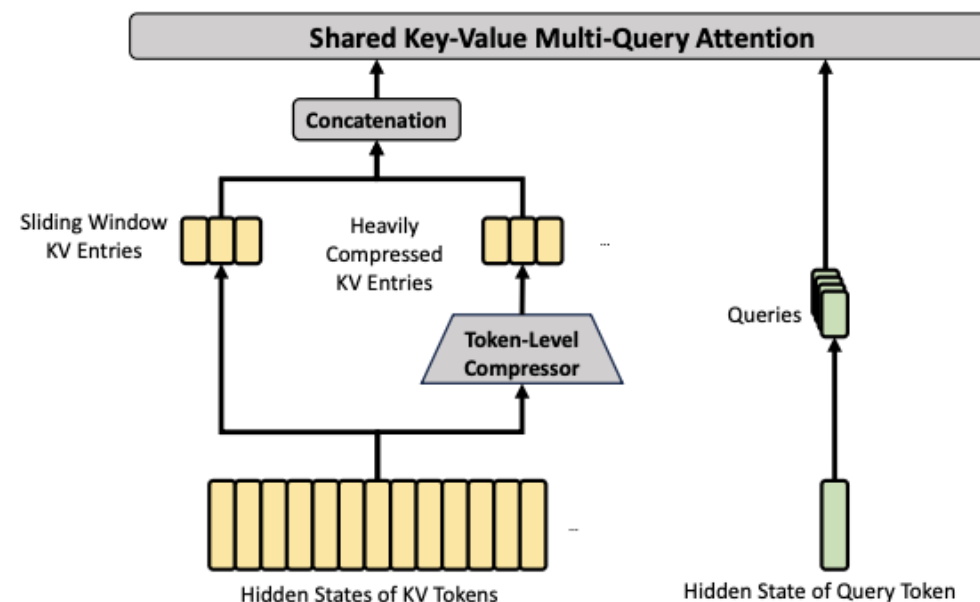
重度压缩注意力 (HCA)

重度压缩注意力 (HCA)

HCA 的核心架构如图所示，它以更重的方式压缩 KV 缓存，但不执行稀疏注意力操作。

✓ 压缩的键值条目。

总体而言，HCA 的压缩策略与 CSA 类似，但使用更大的压缩比例 m' ($m' \gg m$)，并且不执行稀疏注意力操作。



重度压缩注意力 (HCA)

✓ 压缩的键值条目。

具体地，首先， HCA 计算所有原始的 KV 条目 $C \in \mathbb{R}^{m \times k}$ 及其对应的压缩权重 $Z \in \mathbb{R}^{m \times m}$ ：

$$C = H \cdot W^K Z = H \cdot W^Z$$

其中， $W^K, W^Z \in \mathbb{R}^{d \times k}$ 是可训练的参数，且每个压缩的 KV 条目将根据压缩权重和可学习的位置偏移量矩阵 $B \in \mathbb{R}^{m \times k}$ 被压缩到一个新的空间中，生成压缩后的条目 $C_t^{Comp} \in \mathbb{R}^{k \times m}$ ：

$$S_{m(t+1)}^{(i-1)} = \text{SoftmaxRow} \left(Z_{m(t+1)}^{(i-1)} + B \right) C_t^{Comp} = \sum_{i=1}^{n_t} S_i \cdot C_i$$

通过这种压缩操作， HCA 将序列长度压缩为原来的 $\frac{1}{m}$ 倍。

重度压缩注意力 (HCA)

✓ 共享键值 MQA 和分组输出投影。

HCA同样采用了与CSA相似的共享KVMQA和分组输出投影策略。

在执行KV压缩后，对于查询Token，HCA首先以低秩方式生成注意力查询 $\{q_{t,1}, q_{t,2}, \dots, q_{t,n_q}\}$ ：

$$c_t^Q = h_t \cdot W^Q [q_{t,1}, q_{t,2}, \dots, q_{t,n_q}] = c_t^Q \cdot W^Q$$

其中， $h_t \in \mathbb{R}^d$ 是查询Token的输入隐藏状态， n_q 表示查询头的数量， $W^Q \in \mathbb{R}^{d \times m_q}$ 是查询的上投影矩阵。

接下来，对 $\{q_{t,i}\}$ 和 C_t^{Comp} 执行MQA操作：

$$o_t^i = \text{CoreAttn}(\text{query} = q_{t,i}, \text{key} = C_t^{Comp}, \text{value} = C_t^{Comp})$$

其中， $o_t^i \in \mathbb{R}^e$ 是第 i 个头在第 t 个Token上的核心注意力输出。

重度压缩注意力 (HCA)

✓ 共享键值 MQA 和分组输出投影。

接下来, HCA 会将 n_h 输出分成 9 组对于每一组输出 $o_{t,i}^g \in \mathbb{R}^{d_g}$, HCA 将其投影到一个 d_g - 维的中间输出空间:

$$o_{t,i}^g = W^g \cdot o_{t,i}$$

其中, $W^g \in \mathbb{R}^{d_g \times d}$ 是可学习的投影矩阵, 且 $d_g < d$ 。

最后, HCA 将中间输出投影 $o_{t,i}^g \in \mathbb{R}^{d_g}$ 投影到最终的注意力输出 $\delta_t \in \mathbb{R}^d$: $\delta_t = W^\delta \cdot o_{t,i}^g$

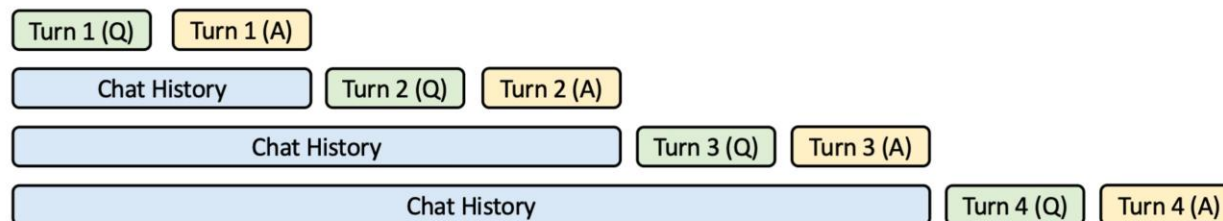
目录

Contents

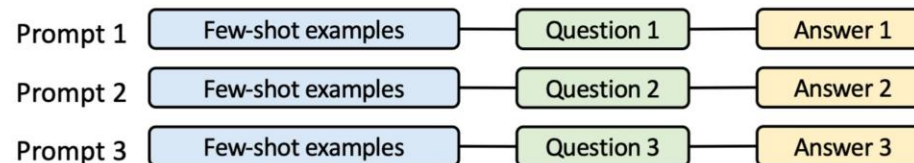
- 1 连续批处理
- 2 PagedAttention 与KV Cache
- 3 RadixAttention 与前缀共享**
- 4 调度优化与 CPU 开销隐藏

机会: KV 缓存重用

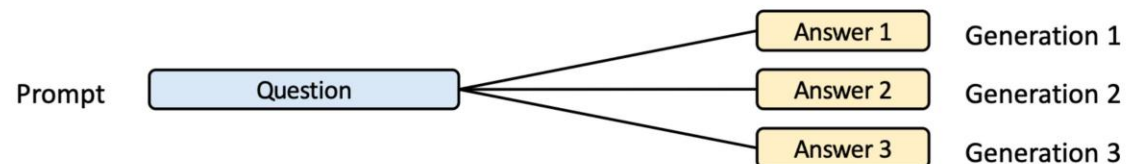
(a) Multi-turn chat



(b) Few-shot learning

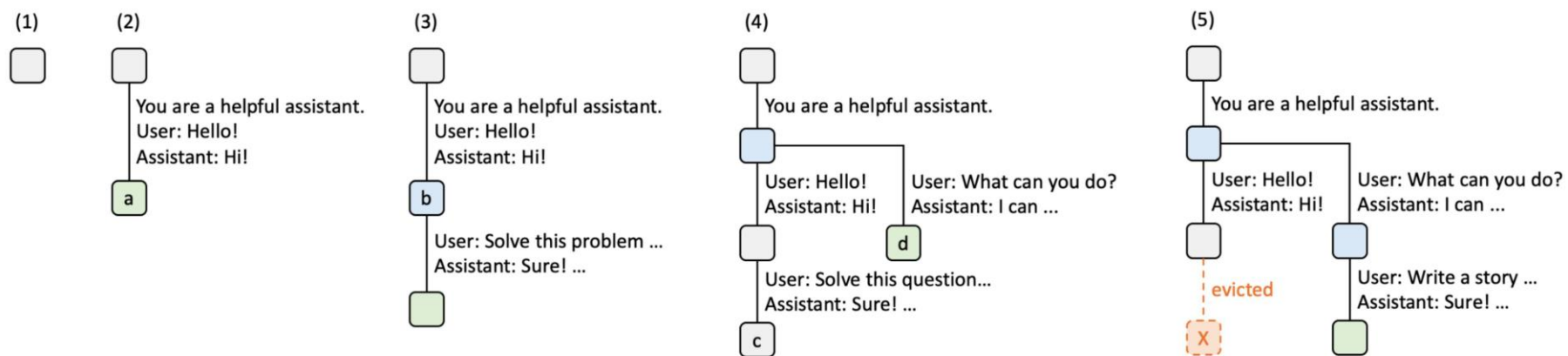


(c) Self-consistency



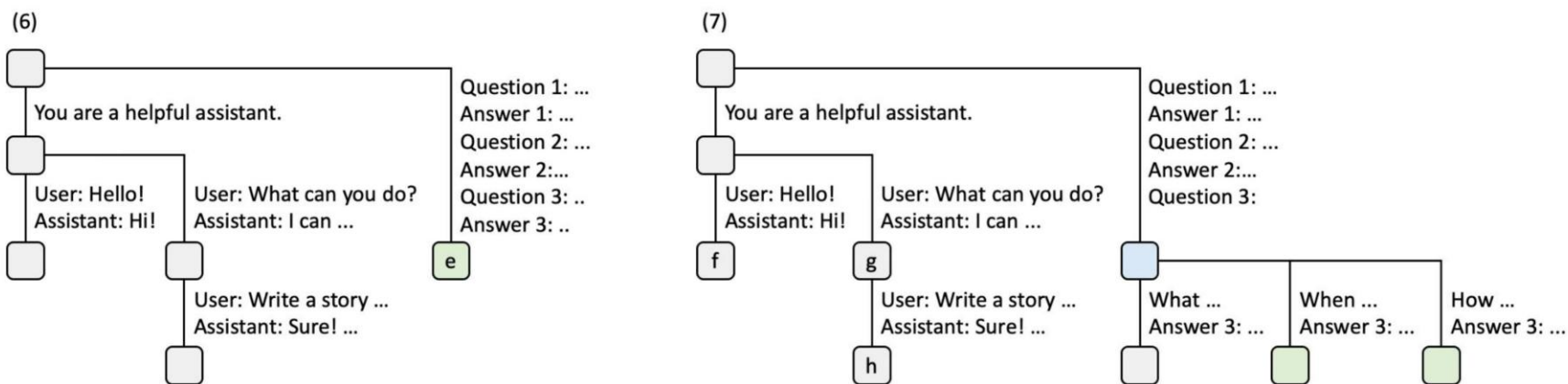
RadixAttention带有前缀共享的注意力

- 现有系统：请求完成后丢弃KV缓存
- RadixAttention：为基树（紧凑前缀树）中的所有请求维护LRU缓存



RadixAttention帶有前綴共享的注意力

- 现有系统：请求完成后丢弃KV缓存
- RadixAttention：在基数树（紧凑前缀树）中维护所有请求的LRU缓存



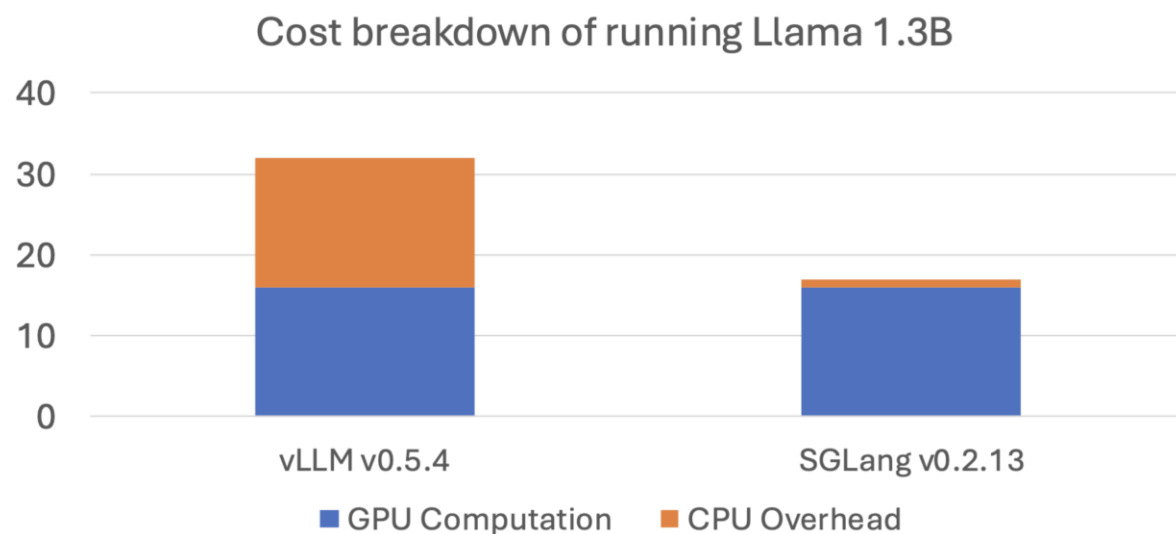
目录

Contents

- 1 连续批处理
- 2 PagedAttention 与KV Cache
- 3 RadixAttention 与前缀共享
- 4 调度优化与 CPU 开销隐藏**

CPU开销隐藏

- 一个未优化的推理引擎在CPU调度上可能浪费超过50%的时间



Source: https://mlsys.wuklab.io/posts/scheduling_overhead/

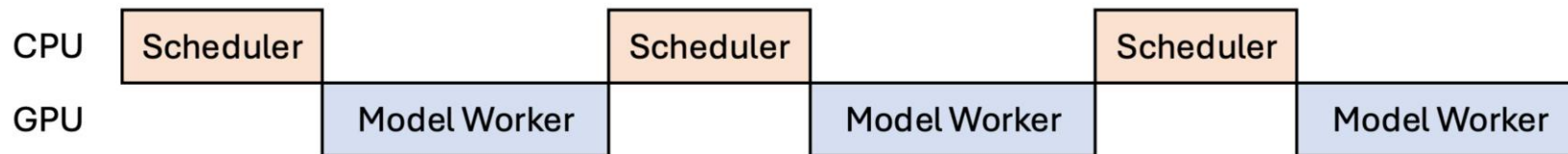
CPU Overhead

CPU调度器迭代执行

1. 接收用户输入的消息
2. 处理模型工作者的结果
3. 检查停止条件
4. 执行前缀匹配和请求重排
5. 为下一批分配内存

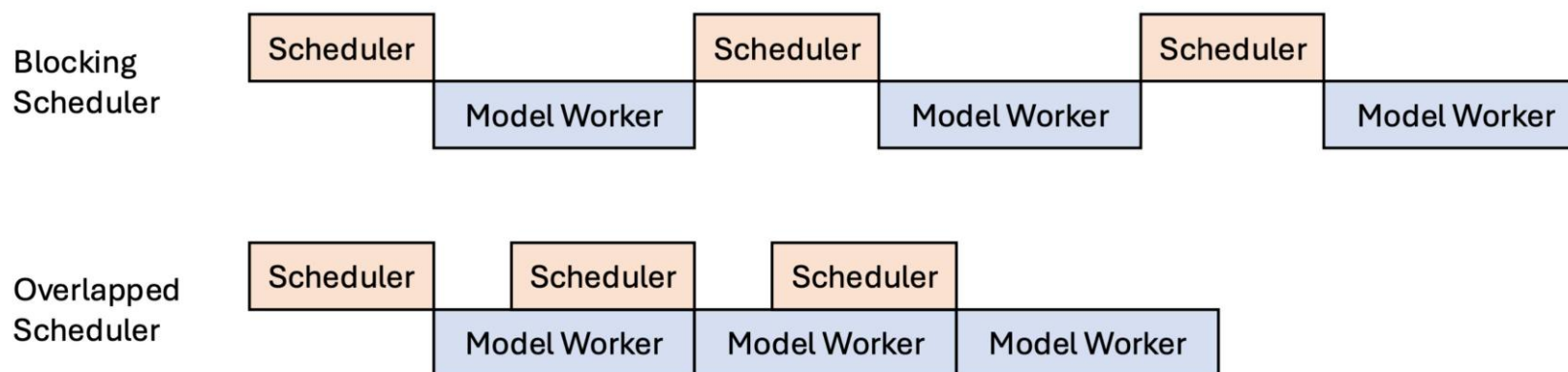
Pseudo code (every line is blocking)

```
while True:
    recv_reqs = recv_requests()
    process_input_requests(recv_reqs)
    batch = get_next_batch_to_run()
    result = run_batch(batch)
    process_batch_result(batch, result)
```



重叠调度器

- 隐藏CPU开销，通过重叠调度器与模型工作者



SGLang中的重叠调度

- 目标：重叠运行 `run_batch` (GPU) 和处理 `batch_result` (CPU)

Normal version

```
while True:
    recv_reqs = recv_requests()
    process_input_requests(recv_reqs)
    batch = get_next_batch_to_run()
    result = run_batch(batch)
    process_batch_result(batch, result)
```

Overlap version

```
last_batch = None
while True:
    recv_reqs = recv_requests()
    process_input_requests(recv_reqs)
    batch = get_next_batch_to_run()
    result = run_batch(batch)
    result_queue.put((batch, result))
    if last_batch is not None:
        tmp_batch, tmp_result = result_queue.get()
        process_batch_result(tmp_batch, tmp_result)
    last_batch = batch
```

`run_batch` is non-blocking and returns a reference;
overlapping `run_batch` of the current iteration with
`process_batch_result` of the previous iteration

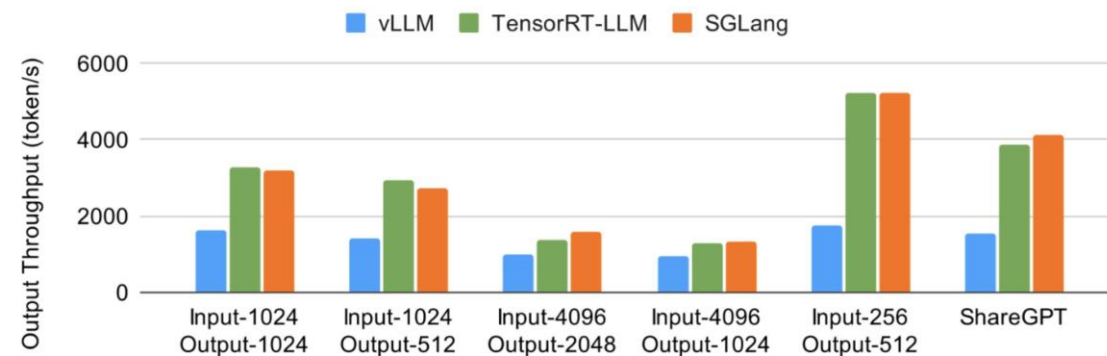
解决依赖

- 核心思想：延迟完成条件检查
- 假设一个请求未完成，并立即在下一个解码批次中运行它
- 通过支付解码一个无用的标记的额外开销来解决依赖关系

* SGLang: Efficient Execution of Structured Language Model Programs

评估

Llama-8B (bf16) on 1 x A100. Higher Throughput is Better.



Llama-70B (fp8) on 8 x H100. Higher Throughput is Better.

